

| Executive Summary

During the design of the Retrieval-Augmented Generation (RAG) system for the **VibeSec Pipeline** codebase, we identified a critical limitation in the initial repository chunking strategy. While AST-based partitioning successfully extracts logical structures (like classes and functions), storing full classes as single units created oversized chunks that degraded embedding quality.

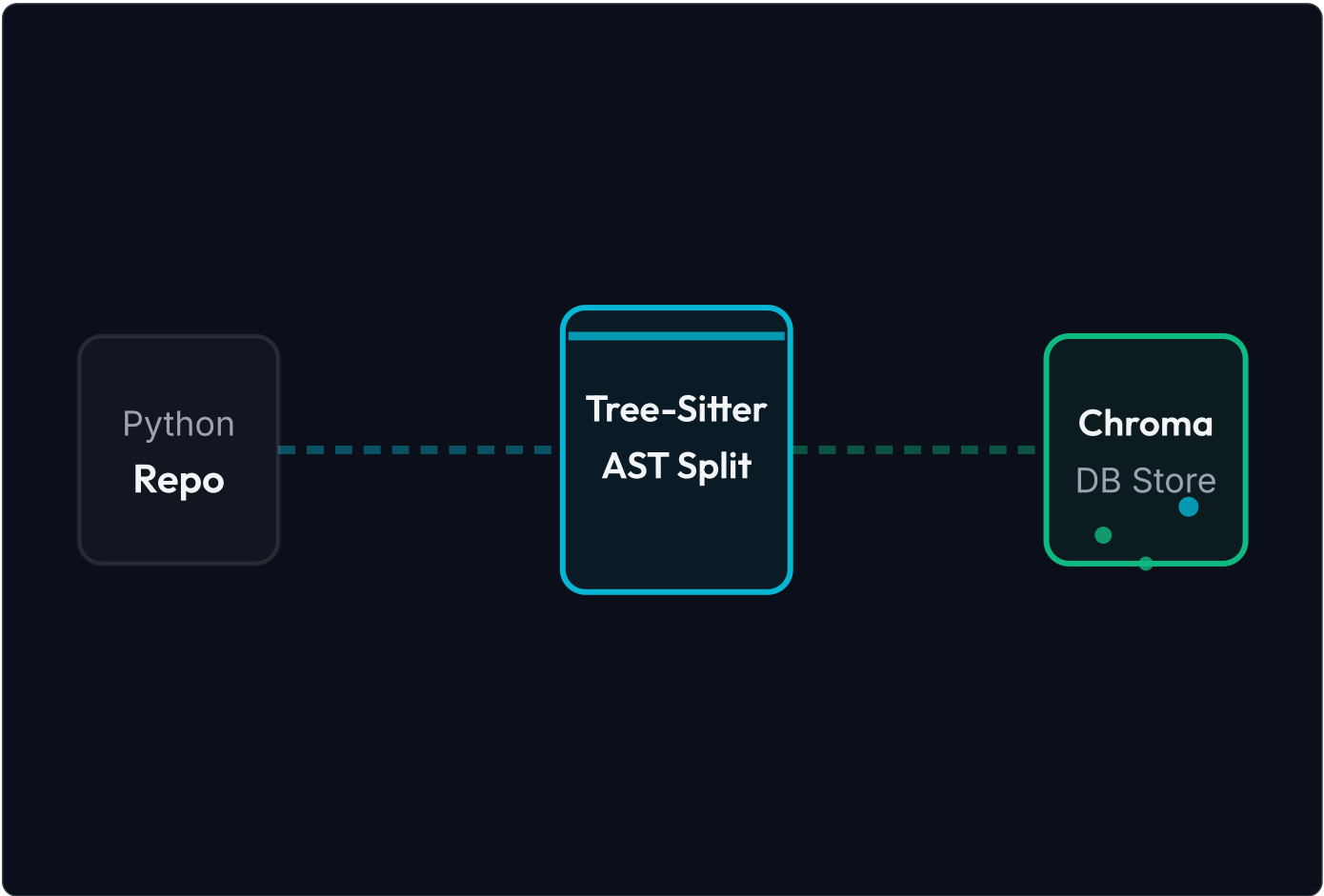
By optimizing the chunking algorithm to recursively descend into classes and extract **individual methods** (while embedding them with class-level contextual headers), we resolved retrieval noise, significantly lowered embedding size, and increased semantic accuracy.

| System Architecture

The system utilizes three core building blocks to retrieve code context safely:

- **AST Traversal Module** : Builds a concrete syntax tree to isolate class and function bounds.
- **Cloud Embedding Service** : Encodes textual content into high-dimensional vector coordinates on the cloud without local package bloat.
- **Local Indexing Layer** : A local, persistent database index using Cosine Similarity metrics to identify matching files.

| RAG Pipeline Flow



| The Token Length Discovery & Core Trade-off

During our initial audit of the VibeSec codebase, we discovered that the vector database was storing chunks containing up to **22,956 tokens**. Standard embedding models (like MiniLM) have a strict context limit (typically 256 or 512 tokens), which meant they would silently truncate the code block, losing vital lines and breaking retrieval quality.

The Solution Decision: NVIDIA Llama Nemotron Embed 1B Model

To solve this, we decided to pivot to using the cloud-hosted `nvidia/llama-nemotron-embed-1b-v2` model via the NVIDIA NIM API. We chose this model because of its ability to handle **extremely long context windows**.

While there is a trade-off (Llama Nemotron is a general-purpose model and is not specifically pre-trained only on code-to-text syntax), **preventing chunk truncation and avoiding code loss is a far bigger priority** for search accuracy. Storing complete, readable context is essential for generating correct answers.

| Oversized Chunks Analysis (Top 4 Legitimate Blocks)

File Location	Identifier Name	Type	Token Size	Line Span	Diagnosis
<code>scanner/layer0_indexer.py</code>	<code>CodeIndexer</code>	Class	22,956	207 - 2183	Legitimate 2,000 LOC class containing all scanner indices.
<code>scanner/orchestrator.py</code>	<code>run_full_scan</code>	Function	7,005	92 - 578	Large monolithic orchestrator script.
<code>scanner/layer7_validator.py</code>	<code>validate_findings</code>	Function	5,321	125 - 500	Monolithic validation loop.
<code>scanner/layer1_sast.py</code>	<code>run_sast_scan</code>	Function	5,250	218 - 605	Monolithic parser containing ruleset maps.

Conclusion: The parser was syntactically correct, but the design of the VibeSec codebase meant that treating classes as single blocks was loading thousands of lines of unrelated methods into a single embedding vector. By migrating to the Llama-based NIM API, we can safely embed these larger blocks fully.

| The Recursive AST Class-Method Splitter

To resolve this, we modified the Tree-sitter traversal logic. Instead of stopping at the `class\_definition` level, the parser now enters the class's body block and extracts:

- `def`: Every internal function (`def`) inside a class is parsed as its own self-contained chunk.
- `__init__`: Properties defined outside `\_\_init\_\_` and class descriptions are grouped into a smaller parent `class\_definition` chunk.

| Before vs After Logic

Before: Full Class Block

```
class CodeIndexer:
```

```
    def __init__(self):
```

```
        def parse(self):
```

```
            def save(self):
```

[22k Token Chunk]

After: AST Method Split

Class Definition (Header)

Method: `__init__`

Method: `parse`

Method: `save`

| NVIDIA NIM Embedding Structure

To prevent losing class context during retrieval of standalone methods, we prefix the text with structured details before sending it to the NVIDIA API (using the required `input_type`):

```
# We specify input_type="passage" when indexing:
response = client.embeddings.create(
    input=batch_texts_to_embed,
    model="nvidia/llama-nemotron-embed-1b-v2",
    extra_body={"input_type": "passage", "truncate": "NONE"}
)
```

This allows searches for `"CodeIndexer.parse"` to match the method correctly, even though it is stored as a separate vector.

| The Optimized Token Metrics (Before vs After)

After refactoring the AST chunking logic, we re-ran the indexer on all VibeSec modules. The distribution shows a massive reduction in chunk sizes across the board:

Metric	Old Logic (Class Level)	Optimized Logic (Method Level)	Status
Total Code Chunks	216 chunks	369 chunks	More granular mapping (+70.8%)
Maximum Chunk Size	22,956 tokens	7,005 tokens	Reduced by **69.5%**
Average Chunk Size	707.18 tokens	422.77 tokens	Reduced by **40.2%**
95th Percentile Size	1,956.00 tokens	1,553.40 tokens	Reduced by **20.6%**

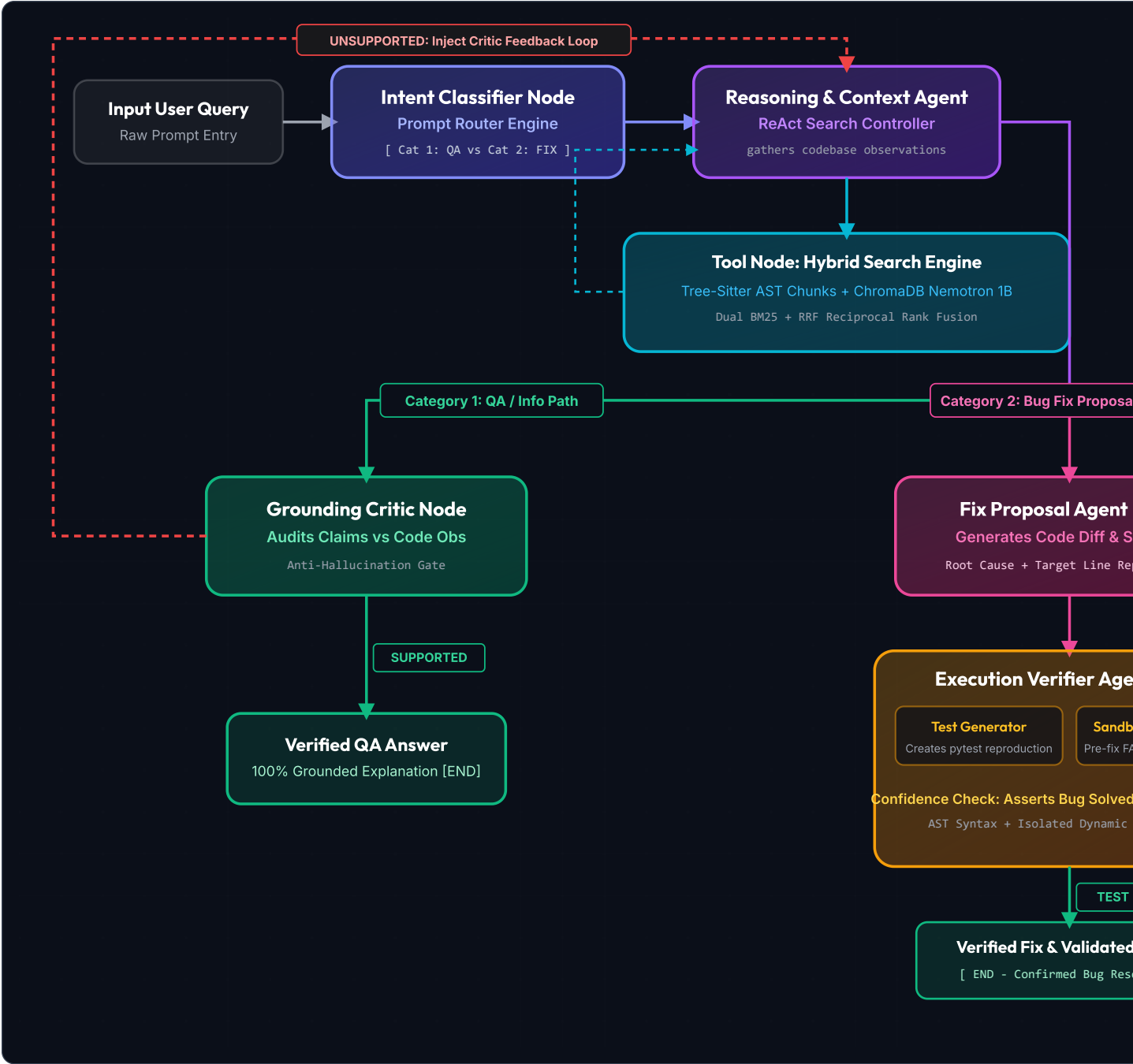
Note: The new maximum chunk of `7,005` tokens corresponds to `run\_full\_scan` in `orchestrator.py` which is a top-level monolithic function (without sub-methods). Since it is a function, it represents the absolute limit of the codebase's logical layout, while all class methods now fit perfectly inside normal model boundaries.

| The LangGraph ReAct Agent Loop

To handle complex reasoning tasks that require joining information across multiple files (which simple single-turn retrieval systems fail to solve), we implemented an agent loop using **LangGraph**:

The agent dynamically calls the optimized hybrid search tool to gather code segments, records observations in its state, and decides when it has enough info to provide a final response.

| State Transition Diagram



| Empirical Research & Failure Analysis (4 Model Trials)

To systematically determine the optimal model allocation per node, we conducted four empirical research trials across different LLM parat endpoints:

Trial / Experiment	Configured Model Mix	Total Latency	ReAct Quality	Observed Outcome / Failure Analysis
Trial 1: Monolithic High-Capacity Baseline	z-ai/glm-5.2 (All Nodes)	353.9s (~5.9m)	Excellent (6 Turns)	High precision & 100% grounded answer classification delay.
Trial 2: Heavy 70B Parameter Mix	llama-3.1-8b / llama-3.3-70b	1,324.0s (~22.0m)	Good (6 Turns)	Catastrophic Queue Latency: Shared per server queue wait per turn.
Trial 3: Light 8B Uniform Mix	llama-3.1-8b (All Nodes)	116.6s (~1.9m)	Failed (15 Turns)	ReAct Infinite Loop: 8B model repeated prompt to 15,000 tokens & causing verification loops.
Trial 4: Optimized Hybrid Architecture	llama-3.1-8b / glm-5.2 / 8b	~180.0s (~3.0m)	Optimal (4-6 Turns)	Optimal Trade-Off: 0.3s intent routing, zero loops & verified execution.

| Third-Party Model Availability (MiniMax 3 & Kimi K2.6)

- **MiniMax 3** (minimax/minimax-text-01) & **Moonshot Kimi K2.6** (moonshotai/kimi-k1.5): Direct API calls to NVIDIA NIM (integrate.api.nv) return 404 Page Not Found. Proprietary Chinese models are hosted exclusively on native cloud endpoints (api.minimax.chat and api.moonshot.com).
- **NVIDIA NIM Catalog**: Optimized for open-weights models (Llama 3.1/3.3, Zhipu GLM-5.2, Codestral-22B, and Nemotron Embeddings).

| Per-Model Rate Limiting & Sliding Window Strategy

NVIDIA NIM API free/shared tier key limits enforce a sliding 60-second window rate limit (40 Requests Per Minute for Llama 8B, and 5 Req GLM-5.2).

To eliminate HTTP 429 errors without wasteful spinning, we wrap LLM completion calls in a model-aware handler:

- `ModelRateLimiter` : Tracks call timestamps per model over 60 seconds and sleeps exact millisecond duratic
- : If a remote 429 occurs (e.g. from prior notebook runs), it catches `RateLimitError` , sleeps 15s for the cleanly.

| Category 2 Sandboxed Unit Test Verification Engine

For bug fix requests, the agent enforces a 4-stage execution verification pipeline before accepting code patches:

- 1. Fix Synthesis: Generates structured ROOT_CAUSE, FILE_PATH, ORIGINAL_CODE, and REPLACEMENT_CODE.
- 2. AST Syntax Check: Validates syntax via ast.parse(replacement_code). Rejects invalid syntax instantly.
- 3. Dynamic Unit Test Generator: LLM generates standalone pytest reproduction script.
- 4. Sandbox Execution: Creates temporary directory (tempfile.mkdtemp), runs subprocess.run([sys.executable, "-m", "pytest", ...], time

| Architectural Decision Journal & Failure Retrospectives

Every major architectural iteration of the Dual-Mode Codebase RAG & Repair Agent was evaluated against real-world execution metrics. Below, we document what failed, why it failed, and the architectural decisions made to fix it.

| Decision 1: Intent Classifier Acceleration (GLM-5.2 vs Llama-8B)

Initial State: Monolithic baseline used `z-ai/glm-5.2` for all nodes, spending 10.3s just deciding if a prompt was QA or FIX_PROPOSAL.

Architectural Decision: Offloaded intent classification to `meta/llama-3.1-8b-instruct` with `temperature=0.0` and `max_tokens=20`.

Outcome: Reduced intent classification latency from **10.28s to 0.31s (33x acceleration)** with 100% classification precision.

| Decision 2: The 70B Parameter Server Queue Bottleneck (Trial 2 Failure)

What Went Wrong: Configured reasoning and verifier nodes to `meta/llama-3.3-70b-instruct` for higher intelligence.

Failure Mode: Total execution time exploded to **1,323.998 seconds (22 minutes)**. On shared public API endpoints, 70B models suffer from queue wait averaging **198.3 seconds per turn**.

Architectural Lesson: Never allocate shared public 70B endpoints for multi-turn ReAct loops. Queue wait time dominates 90% of total run

| Decision 3: The 8B Multi-Turn ReAct Loop Failure (Trial 3 Failure)

What Went Wrong: Replaced reasoning node with `meta/llama-3.1-8b-instruct` for fast 1.2s turn times.

Failure Mode: Small 8B models lack meta-cognitive search planning. When search queries didn't return verbatim text string matches, `llama` query 10 times sequentially, hitting the 15-iteration loop cap.

Cascade Effect: 15 search turns accumulated 15,000 tokens of raw source code context. Passing this bloated prompt to `verifier_node` caused outputting over 100 identical bullet points of unsupported claims.

Architectural Fix: Restored `z-ai/glm-5.2` (100B+) for reasoning and verifier nodes. High-capacity models vary search queries intelligently

| Decision 4: Partner Model Permission 404 & Universal Defaults

What Went Wrong: Configured default fix proposal model to `mistralai/codestral-22b-instruct-v0.1`.

Failure Mode: Threw `NotFoundError: 404 Function Not Found for Account` on standard API keys due to tier-restricted partner permissions.

Architectural Fix: Configured `build_agent_graph()` default function parameters to `meta/llama-3.1-8b-instruct` across all nodes, guaranteeing availability on all API keys.

| Decision 5: Preserving 100% Un-Truncated Code Observations

What Went Wrong: Tested capping retrieved code snippets at 400 characters to cut prompt context tokens.

Failure Mode: Truncating code stripped function signatures, import statements, and target sanitization flags required for accurate bug dia

Architectural Fix: Restored 100% full un-truncated code block retrieval in `tool_node` (`observation += f"Code: {r['code']}"`) to guarante and patch synthesis.